1 Problem

The health care department has to hire new doctors. In the department there are m specializations (orthopedics, pediatrics, etc.). For each specialization j a minimum number of new doctors b_j are required. There are n candidate doctors. Each doctor i can cover several specializations (at the same time). S_i denotes the set of specializations that doctor i covers. The salary cost of each doctor i, if hired, is c_i.

The problem of the health care department manager is to hire a subset of doctors that guarantees to cover all requirements at minimum cost.

1.1 Formulation

To represent the problem we use a bipartite graph. One side of the graph will be the set of doctors D (indexed by i), while on the other the set of all specializations S (indexed by j). The set of arcs between the two sets will be A, which will be defined in terms of the parameter S_i as such:

$$A = \{(i, j) | i \in D \wedge j \in S_i\}$$

Thus the resulting graph will be $G = (D \times S, A)$.

As parameters we will use the aforementioned S_i representing the set of specialization of doctor i, the salaries of each doctor c_i and the minimum hires of each specialization b_j.

As variables we will use two binary variables:

1. x_{ij} indicates whether the edge (i, j) has been chosen, this translates to whether the doctor i has been chosen to cover specialty j;
2. y_i indicates whether a doctor has been hired to work for any specialty

The constraints and objective are thus as follows:

$$\min \sum_{i \in D} c_i y_i \tag{1}$$

$$\sum_{(i,j) \in BS(j)} x_{ij} \geq b_j \qquad \forall j \in S \tag{2}$$

$$y_i - \sum_{j \in S} x_{ij} \geq 0 \qquad \forall i \in D \tag{3}$$

$$x_{ij} \in \{0, 1\}, \quad y_i \in \{0, 1\} \tag{4}$$

Where $BS(j)$ is the backwards star of node j. Equation (2) represents the doctor requirement constraint while equation (3) the linking constraint for y_i.

1.2 Some python code

Some python code has been write and can be run in a python notebook using `mip`.

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import mip

%matplotlib inline

np.random.seed(4321)
n = 5
m = 3

### Sets and parameters
D = [i for i in range(n)]
```

```python
S = [j for j in range(m)]

Si = {i: set(np.random.randint(0, m, size=3)) for i in D}
C = [np.random.randint(1, 10) for i in D]
A = [(i, j) for i in D for j in Si[i]]

B = np.random.randint(1, max([len(Si[i]) for i in Si]), size=m)

m = mip.Model()
### Variables
vars = {(i, j): m.add_var(var_type=mip.BINARY) for (i, j) in A} # x_ij
selected = [m.add_var(var_type=mip.BINARY) for i in D]          # y_i

### Constraints
for j in S:
    m.add_constr(mip.xsum(vars[i, j] for (i, j1) in A if j == j1) >= B[j])
for i in D:
    m.add_constr(selected[i] >= mip.xsum(vars[i,j] for (i1, j) in A if i1 == i))

### Objective
m.objective = mip.minimize(mip.xsum(C[i] * selected[i] for i in D))
m.optimize()
```

2 Problem variant

Different from what has been outlined in 1, we can now leave some specialization uncovered and activate interregional contracts. For each specialization the cost of the contract if t_j. In case the contract is activated, there is no need to hire any doctor to cover b_j positions. Our objective now is to minimize the total cost of both new hires and contracts.

2.1 Formulation

We can keep most of what we built in 1.1. However, we need to add some ingredients in light of the new requirements:

Parameters We add a new parameter t_j indicating the cost of the contract for specialization j.

Variables We add a new binary variable z_j indicating whether a contract for specialization j has been chosen.

We can the rewrite the objective and constraints as follows:

$$\min \sum_{i \in D} c_i y_i + \sum_{j \in S} t_j z_j \tag{5}$$

$$\sum_{(i,j) \in BS(j)} x_{ij} \geq b_j (1 - z_j) \qquad \forall j \in S \tag{6}$$

$$y_i - \sum_{j \in S} x_{ij} \geq 0 \qquad \forall i \in D \tag{7}$$

$$x_{ij} \in \{0,1\}, \quad y_i \in \{0,1\}, \quad z_j \in \{0,1\} \tag{8}$$

Where (6) is the new doctor requirement constraint while (7) is the same linking constraint as before.

2.2 Some other python code

Like before, some more python code is provided so the model can be tested inside a python notebook. It is simply a modified version of the previous listing.

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import mip

%matplotlib inline

np.random.seed(4321)
n = 5
m = 3

### Sets and parameters
D = [i for i in range(n)]
S = [j for j in range(m)]

Si = {i: set(np.random.randint(0, m, size=3)) for i in D}
C = [np.random.randint(1, 10) for i in D]
T = [np.random.randint(5, 15) for j in S]

A = [(i,j) for i in D for j in Si[i]]

B = np.random.randint(1, max([len(Si[i]) for i in Si]), size=m)

m = mip.Model()
### Variables
vars = {(i,j): m.add_var(var_type=mip.BINARY) for (i,j) in A} # x_ij
selected = [m.add_var(var_type=mip.BINARY) for i in D]        # y_i
contract = [m.add_var(var_type=mip.BINARY) for j in S]        # z_j

### Constraints
for j in S:
    m.add_constr(
        mip.xsum(vars[i, j] for (i, j1) in A if j == j1) >= B[j]*(1 - contract[j]))
for i in D:
    m.add_constr(selected[i] >= mip.xsum(vars[i,j] for (i1,j) in A if i1 == i))

### Objective
m.objective = mip.minimize(
    mip.xsum(C[i]*selected[i] for i in D) + mip.xsum(T[j]*contract[j] for j in S))
m.optimize()
```